Evolutionary Computation lab

Problem description

Problem that we are using to benchmark different algorithms is modified version of **TSP**.

In this problem each nodes are placed in euclidean space, with additional cost for visiting each one of them.

Our goal is to visit half (rounded up) nodes, and go back to the start is such way that the travel cost is minimized (calculating euclidean distance and with added cost)

Data

	X coordinate	Y coordinate	cost
0	1355	1796	496
1	2524	387	414
2	2769	430	500
3	3131	1199	1133
4	661	87	903
5	628	974	1081
6	1748	185	1859
7	3650	1882	945
8	3474	1230	1389
9	2662	1405	776

Evolutionary Computation lab

Dawid Siera id:156044

Anatol Kaczmarek id:156038

Source code is available on https://github.com/tole-k/evolutionary-computation.git

Problem description

Problem that we are using to benchmark different algorithms is modified version of **TSP**.

In this problem each nodes are placed in euclidean space, with additional cost for visiting each one of them.

Our goal is to visit half (rounded up) nodes, and go back to the start is such way that the travel cost is minimized (calculating euclidean distance and with added cost)

Regret algorithms

NN to any point 2 regret

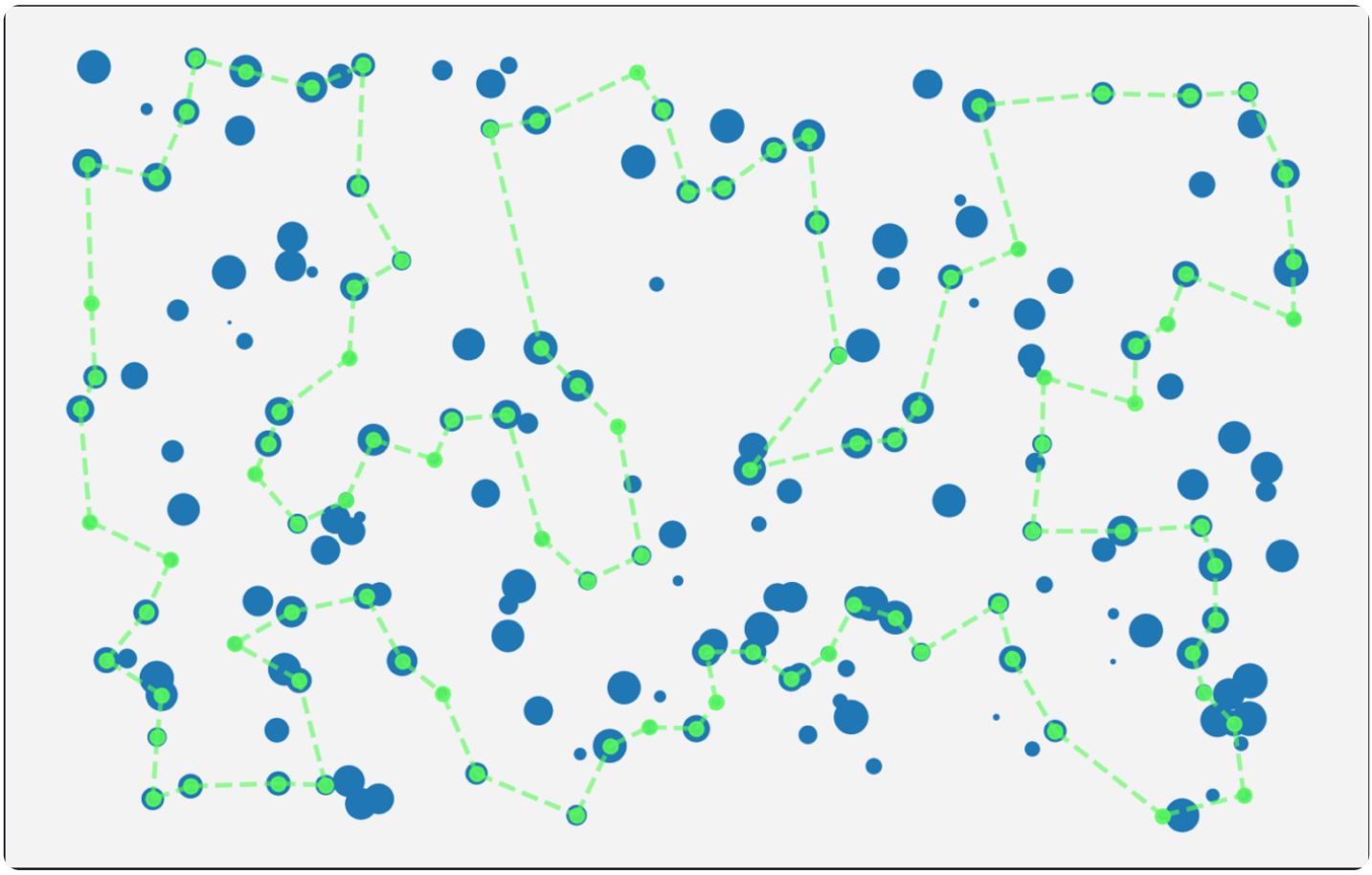
Pseudocode

```
nn_to_any_2_regret_pass(path, candidate_point, distance_matrix):
    best_pos = 0
    min_cost = infinity

    second_best_pos = 0
    second_min_cost = infinity

    for pos from 0 to length(path):
```

```
        if pos == 0:
            cost = distance_matrix[path[0], candidate_point] + candidate_point.co

        else if pos == length(tsp_path):
            cost = distance_matrix[path[length(tsp_path) - 1], candidate_point] +

        else:
            a = tsp_path[pos - 1]
            b = tsp_path[pos]
            cost =
                distance_matrix[a, candidate_point] +
                distance_matrix[candidate_point, b] -
                distance_matrix[a, b] +
                candidate_point.cost

        if cost < min_cost:
            second_best_pos = best_pos;
            second_min_cost = min_cost;
            best_pos = pos
            min_cost = cost
        else if cost < second_min_cost:
            second_best_pos = pos
            second_min_cost = cost

    return best_pos, min_cost, second_min_cost - min_cost

nn_to_any_weighted_2_regret(points, starting_point, distance_matrix, weights):
    tsp_path = [starting_point]
    not_visited_points = points
    remove starting_point from not_visited_points

    for i from 1 to ceil(length(points) / 2):
        insert_spot = 0
        best_point = 0
        min_cost = infinity

        for each candidate_point in not_visited_points:
            pos, cost, regret = nn_to_any_2_regret_pass(tsp_path, candidate_point
            cost = cost * weights[0] - regret * weights[1]
            if cost < min_cost:
                min_cost = cost
                insert_spot = pos
                best_point = candidate_point
```

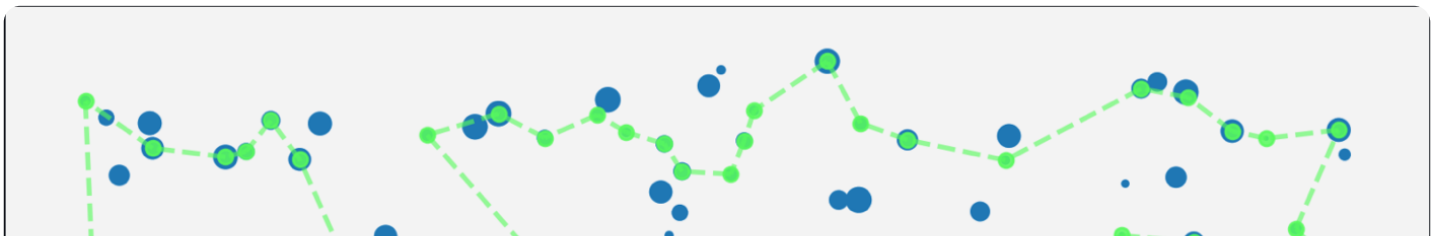
```
        insert closest_point into tsp_path at insert_spot
        remove closest_point from not_visited_points

    return tsp_path
```

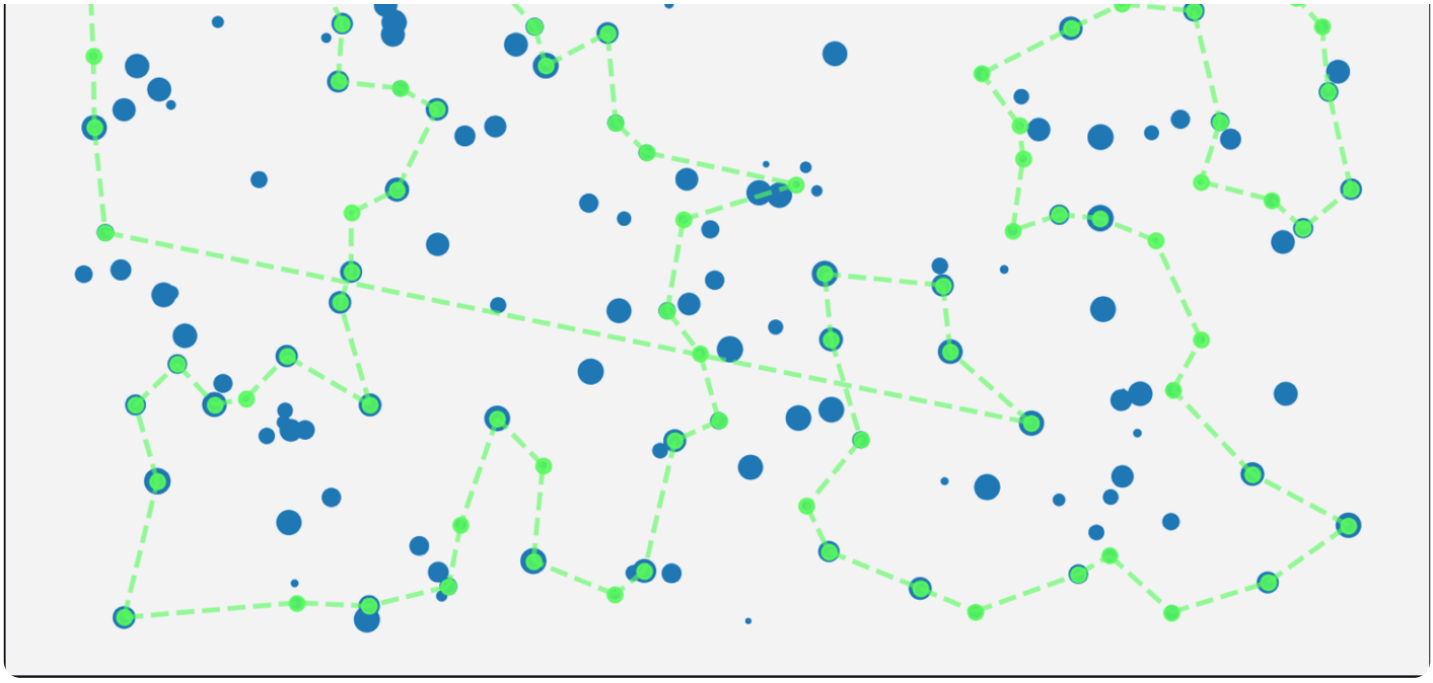
TSP A



Best found path: [16, 175, 56, 31, 38, 157, 17, 196, 91, 57, 52, 106, 185, 8, 165, 39, 90, 27, 71, 164, 7, 21, 132, 14, 102, 128, 167, 111, 130, 148, 15, 64, 114, 186, 23, 89, 183, 153, 0, 141, 66, 176, 79, 133, 151, 109, 118, 59, 197, 116, 43, 42, 5, 96, 115, 198, 46, 68, 93, 36, 67, 108, 69, 199, 20, 22, 146, 103, 34, 160, 48, 30, 104, 177, 10, 190, 4, 112, 35, 184, 166, 131, 24, 123, 127, 70, 6, 154, 158, 53, 136, 121, 100, 97, 152, 87, 2, 129, 82, 25]

TSP B





Best found path: [129, 119, 159, 37, 41, 81, 77, 97, 146, 187, 165, 127, 137, 75, 93, 76, 194, 166, 86, 110, 128, 124, 62, 18, 34, 174, 183, 9, 99, 185, 179, 172, 57, 66, 47, 148, 23, 20, 59, 28, 4, 152, 184, 155, 84, 3, 15, 145, 13, 132, 169, 188, 6, 150, 147, 134, 2, 43, 139, 11, 0, 33, 104, 8, 82, 87, 79, 36, 7, 177, 123, 5, 78, 162, 80, 108, 196, 42, 156, 30, 117, 151, 173, 19, 112, 121, 116, 98, 51, 125, 191, 178, 10, 133, 44, 72, 40, 63, 92, 38]

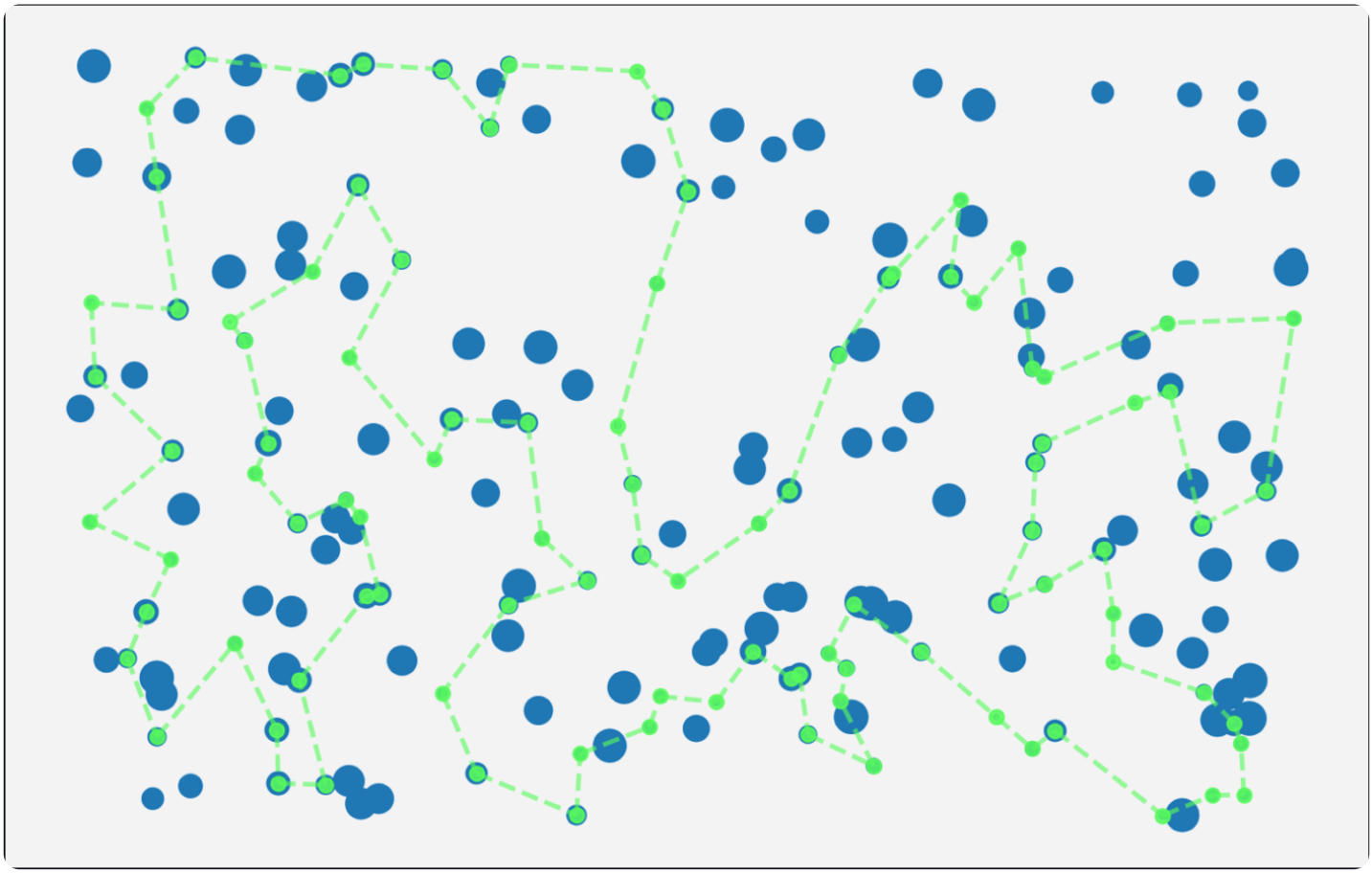
NN to any point weighted 2 regret

Pseudocode

```
nn_to_any_2_regret_pass(path, candidate_point, distance_matrix):
    best_pos = 0
    min_cost = infinity

    second_best_pos = 0
    second_min_cost = infinity

    for pos from 0 to length(path):

        if pos == 0:
            cost = distance_matrix[path[0], candidate_point] + candidate_point.co

        else if pos == length(tsp_path):
            cost = distance_matrix[path[length(tsp_path) - 1], candidate_point] +
```

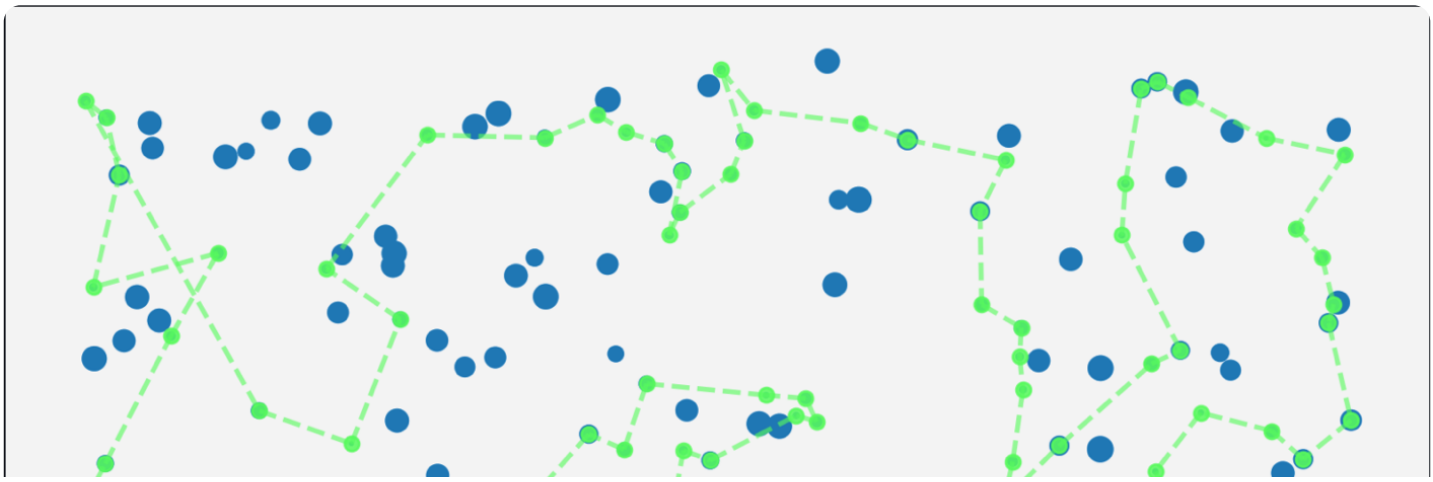
```
        else:
            a = tsp_path[pos - 1]
            b = tsp_path[pos]
            cost =
                distance_matrix[a, candidate_point] +
                distance_matrix[candidate_point, b] -
                distance_matrix[a, b] +
                candidate_point.cost

        if cost < min_cost:
            second_best_pos = best_pos;
            second_min_cost = min_cost;
            best_pos = pos
            min_cost = cost
        else if cost < second_min_cost:
            second_best_pos = pos
            second_min_cost = cost

    return best_pos, min_cost, second_min_cost - min_cost

nn_to_any_weighted_2_regret(points, starting_point, distance_matrix, weights):
    tsp_path = [starting_point]
    not_visited_points = points
    remove starting_point from not_visited_points

    for i from 1 to ceil(length(points) / 2):
        insert_spot = 0
        best_point = 0
        min_cost = infinity

        for each candidate_point in not_visited_points:
            pos, cost, regret = nn_to_any_2_regret_pass(tsp_path, candidate_point
            cost = cost * weights[0] - regret * weights[1]
            if cost < min_cost:
                min_cost = cost
                insert_spot = pos
                best_point = candidate_point

        insert closest_point into tsp_path at insert_spot
        remove closest_point from not_visited_points

    return tsp_path
```

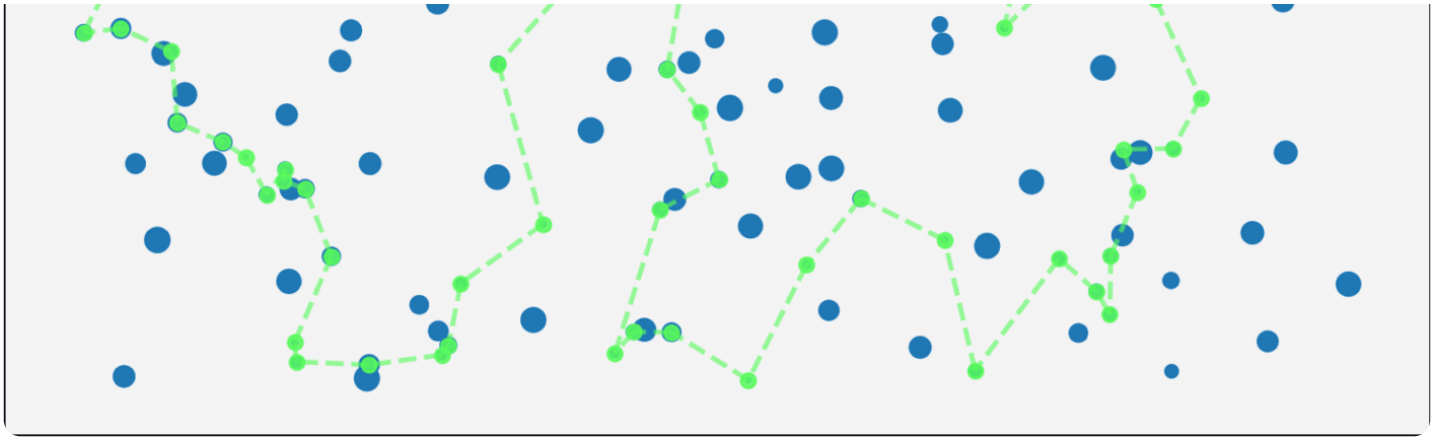
TSP A



Best found path: [108, 18, 199, 159, 22, 146, 181, 34, 160, 48, 54, 177, 184, 84, 4, 112, 35, 131, 149, 65, 116, 43, 42, 5, 41, 193, 139, 68, 46, 115, 59, 118, 51, 151, 133, 162, 123, 127, 70, 135, 154, 180, 53, 121, 100, 26, 86, 75, 101, 1, 97, 152, 2, 120, 44, 25, 16, 171, 175, 113, 56, 31, 78, 145, 179, 92, 129, 57, 55, 52, 185, 40, 196, 81, 90, 165, 106, 178, 14, 49, 102, 144, 62, 9, 148, 124, 94, 63, 79, 80, 176, 137, 23, 89, 183, 143, 0, 117, 93, 140]

TSP B





Best found path: [131, 121, 51, 90, 147, 6, 188, 169, 132, 13, 168, 195, 145, 15, 70, 3, 155, 184, 152, 170, 34, 55, 18, 62, 124, 106, 128, 95, 130, 183, 140, 4, 149, 28, 20, 60, 148, 47, 94, 66, 57, 172, 179, 185, 86, 166, 194, 176, 113, 103, 127, 89, 163, 187, 153, 81, 77, 141, 91, 61, 36, 21, 82, 8, 104, 33, 160, 0, 35, 109, 29, 11, 138, 182, 25, 177, 5, 78, 175, 162, 80, 190, 136, 73, 31, 54, 193, 117, 198, 156, 1, 16, 27, 38, 135, 122, 63, 100, 107, 40]

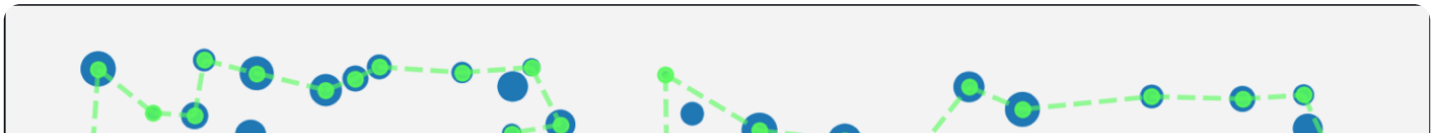
Greedy Cycle 2 regret

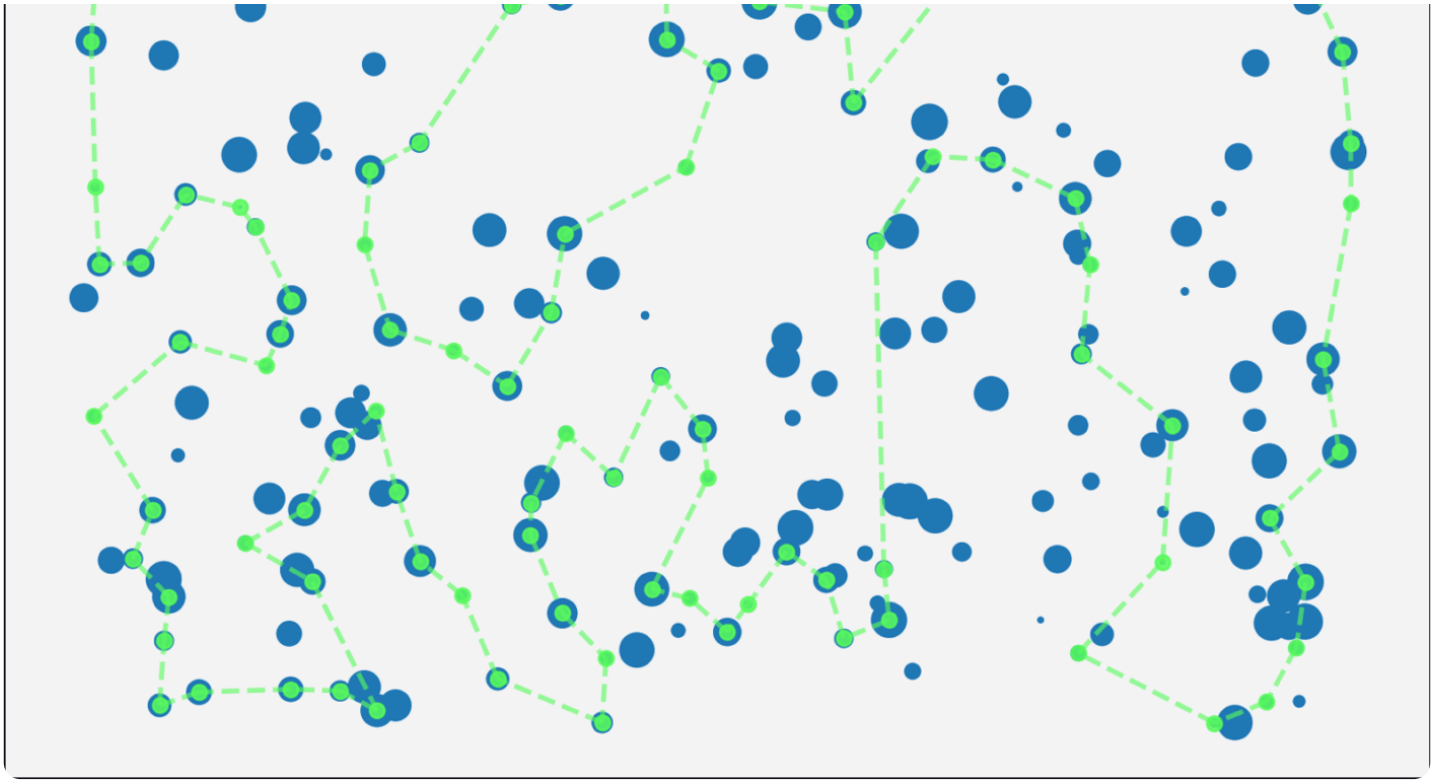
Pseudocode

```
greedy_cycle_2_regret_pass(path, candidate_point, distance_matrix):
    best_pos = 0
    min_cost = infinity

    second_best_pos = 0
    second_min_cost = infinity

    for pos from 0 to length(path):

        if pos == 0 or pos == length(tsp_path):
            a = tsp_path[length(tsp_path) - 1]
            b = tsp_path[0]
        else:
            a = tsp_path[pos - 1]
            b = tsp_path[pos]

        cost =
            distance_matrix[a, candidate_point] +
            distance_matrix[candidate_point, b] -
```

```
            distance_matrix[a, b] +
            candidate_point.cost

        if cost < min_cost:
            second_best_pos = best_pos;
            second_min_cost = min_cost;
            best_pos = pos
            min_cost = cost
        else if cost < second_min_cost:
            second_best_pos = pos
            second_min_cost = cost

    return best_pos, min_cost, second_min_cost - min_cost

greedy_cycle_weighted_2_regret(points, starting_point, distance_matrix, weights):
    tsp_path = [starting_point]
    not_visited_points = points
    remove starting_point from not_visited_points

    for i from 1 to ceil(length(points) / 2):
        insert_spot = 0
        best_point = 0
        min_cost = infinity

        for each candidate_point in not_visited_points:
            pos, cost, regret = greedy_cycle_2_regret_pass(tsp_path, candidate_po
            cost = cost * weights[0] - regret * weights[1]
            if cost < min_cost:
                min_cost = cost
                insert_spot = pos
                best_point = candidate_point

        insert closest_point into tsp_path at insert_spot
        remove closest_point from not_visited_points

    return tsp_path
```

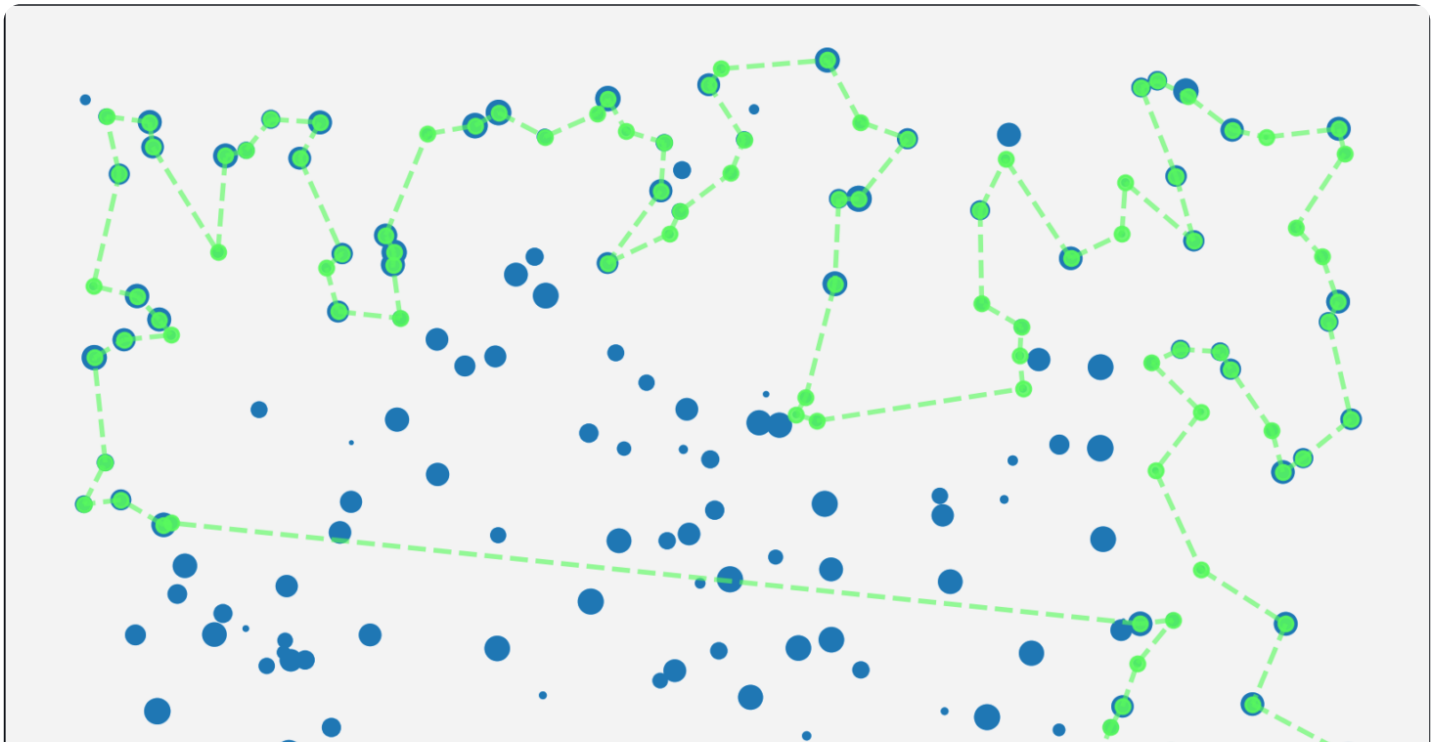
TSP A





Best found path: [1, 150, 86, 100, 121, 53, 158, 180, 173, 63, 122, 80, 133, 151, 162, 161, 194, 135, 70, 127, 123, 24, 149, 65, 77, 166, 184, 35, 156, 112, 4, 190, 10, 177, 104, 54, 48, 34, 181, 42, 5, 96, 41, 193, 159, 195, 146, 22, 20, 134, 18, 69, 108, 67, 36, 140, 93, 117, 143, 153, 0, 46, 198, 115, 197, 59, 72, 51, 141, 137, 23, 76, 183, 83, 64, 15, 73, 132, 21, 7, 164, 71, 27, 90, 187, 98, 157, 188, 113, 171, 16, 44, 78, 91, 55, 106, 32, 102, 62, 148]

TSP B



Best found path: [1, 197, 16, 27, 38, 92, 102, 135, 32, 96, 63, 100, 107, 17, 72, 122, 44, 133, 10, 115, 178, 191, 90, 125, 51, 120, 67, 71, 147, 192, 150, 6, 188, 65, 169, 132, 126, 43, 168, 195, 145, 15, 161, 70, 84, 155, 184, 167, 189, 69, 109, 0, 35, 62, 18, 55, 34, 170, 152, 174, 183, 140, 9, 199, 4, 149, 28, 59, 20, 23, 60, 148, 47, 154, 66, 57, 172, 52, 179, 22, 99, 130, 95, 185, 86, 166, 48, 76, 93, 75, 137, 114, 127, 165, 89, 103, 26, 113, 194, 88]

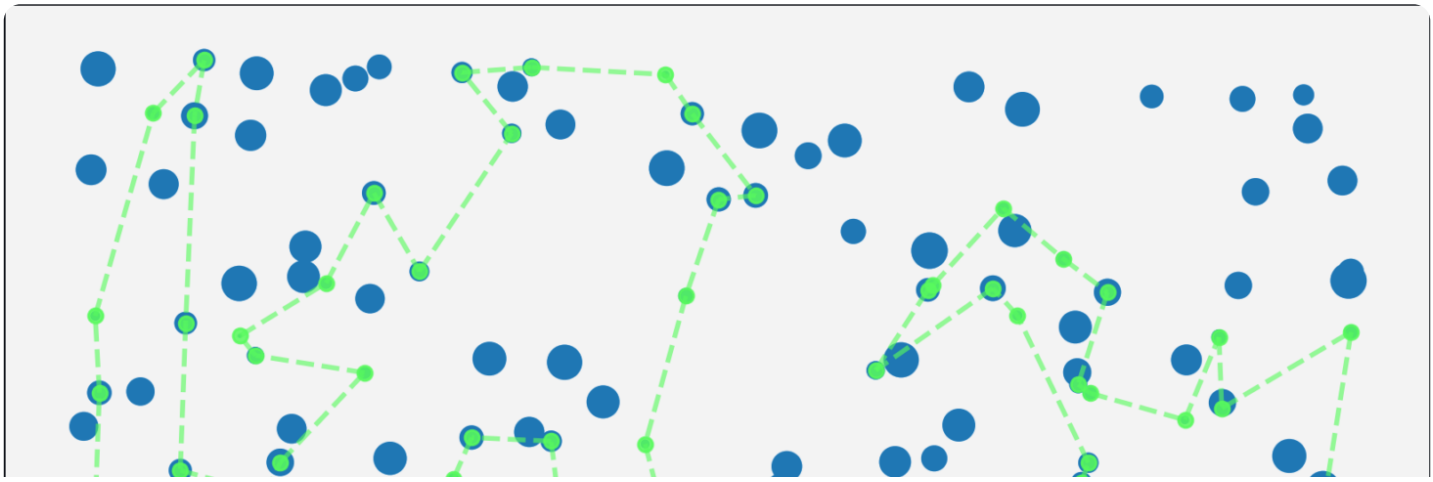
Greedy Cycle weighted 2 regret

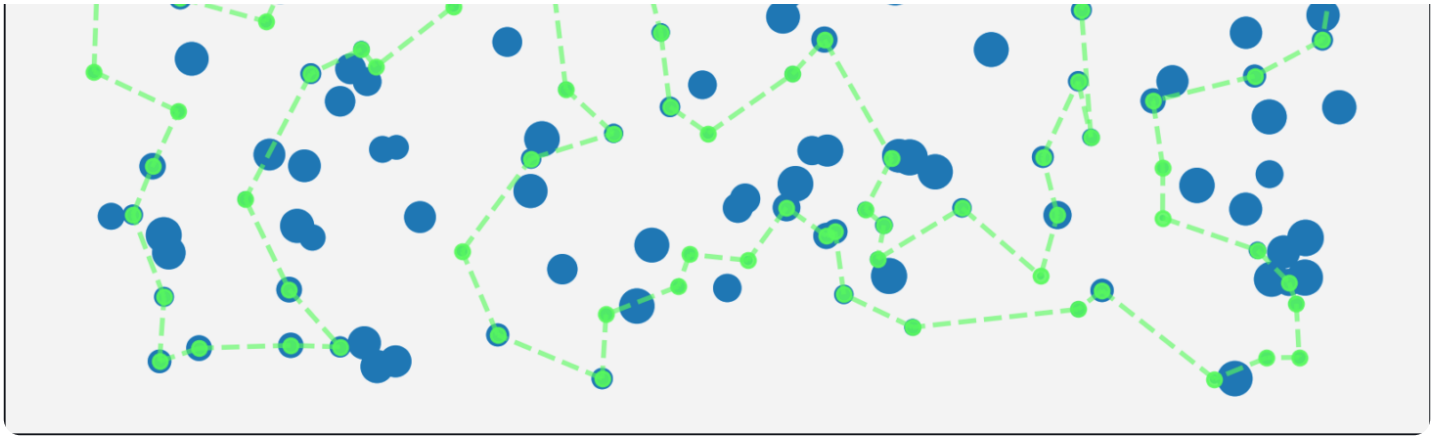
Pseudocode

```
greedy_cycle_2_regret_pass(path, candidate_point, distance_matrix):
    best_pos = 0
    min_cost = infinity

    second_best_pos = 0
    second_min_cost = infinity

    for pos from 0 to length(path):

        if pos == 0 or pos == length(tsp_path):
            a = tsp_path[length(tsp_path) - 1]
            b = tsp_path[0]
        else:
            a = tsp_path[pos - 1]
            b = tsp_path[pos]

        cost =
            distance_matrix[a, candidate_point] +
            distance_matrix[candidate_point, b] -
            distance_matrix[a, b] +
            candidate_point.cost

        if cost < min_cost:
            second_best_pos = best_pos;
            second_min_cost = min_cost;
            best_pos = pos
```

```
            min_cost = cost
        else if cost < second_min_cost:
            second_best_pos = pos
            second_min_cost = cost

    return best_pos, min_cost, second_min_cost - min_cost

greedy_cycle_weighted_2_regret(points, starting_point, distance_matrix, weights):
    tsp_path = [starting_point]
    not_visited_points = points
    remove starting_point from not_visited_points

    for i from 1 to ceil(length(points) / 2):
        insert_spot = 0
        best_point = 0
        min_cost = infinity

        for each candidate_point in not_visited_points:
            pos, cost, regret = greedy_cycle_2_regret_pass(tsp_path, candidate_po
            cost = cost * weights[0] - regret * weights[1]
            if cost < min_cost:
                min_cost = cost
                insert_spot = pos
                best_point = candidate_point

        insert closest_point into tsp_path at insert_spot
        remove closest_point from not_visited_points

    return tsp_path
```

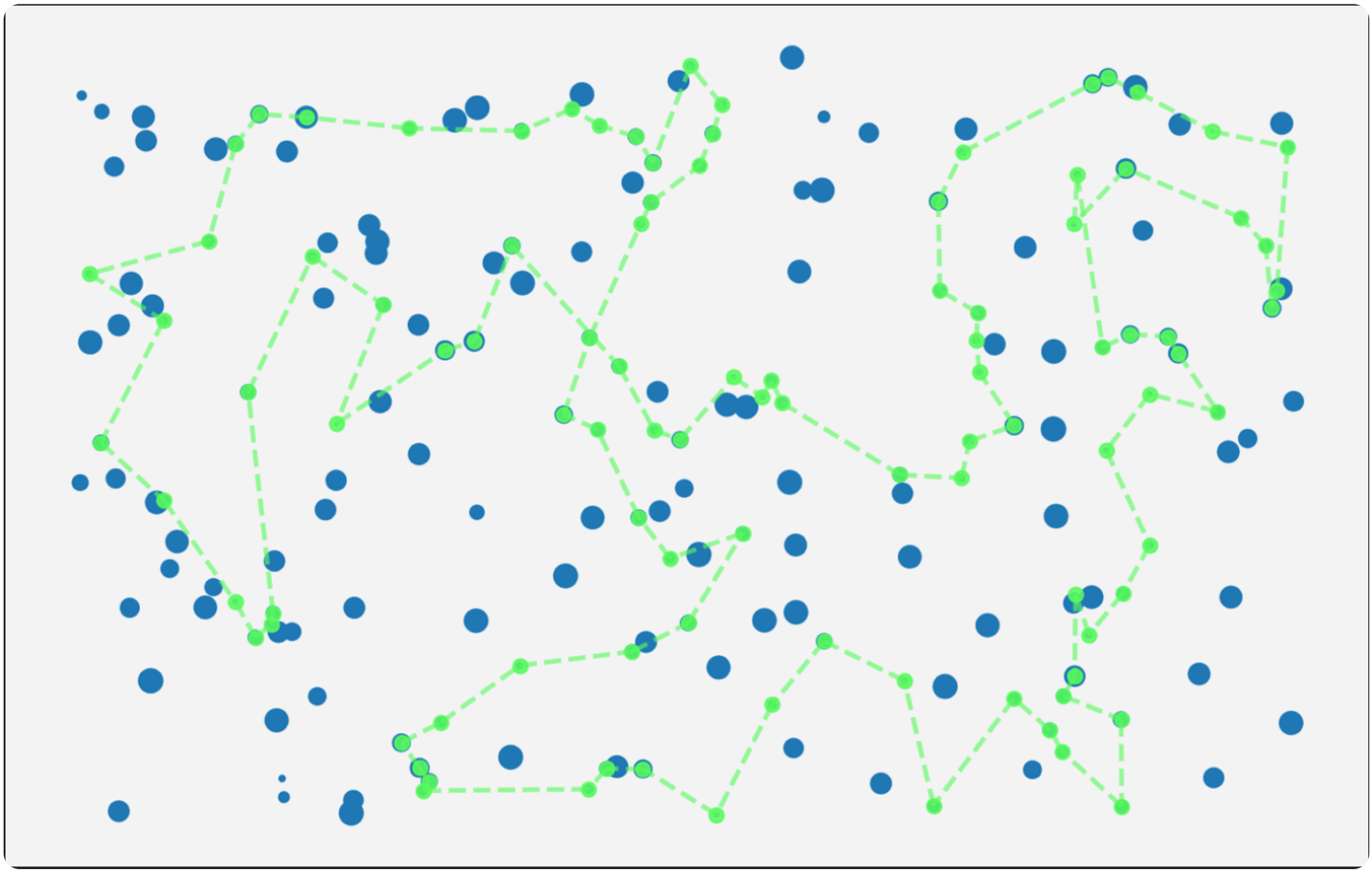
TSP A





Best found path: [137, 176, 80, 79, 63, 94, 124, 152, 97, 1, 101, 2, 120, 82, 129, 57, 92, 55, 52, 49, 102, 148, 9, 62, 144, 14, 138, 178, 106, 185, 165, 40, 90, 81, 196, 179, 145, 78, 31, 56, 113, 175, 171, 16, 25, 44, 75, 86, 26, 100, 121, 53, 180, 154, 135, 70, 127, 123, 162, 133, 151, 51, 118, 59, 65, 116, 43, 184, 84, 112, 4, 190, 10, 177, 54, 48, 160, 34, 146, 22, 18, 108, 69, 159, 181, 42, 5, 115, 41, 193, 139, 68, 46, 0, 117, 143, 183, 89, 186, 23]

TSP B



Best found path: [183, 140, 95, 130, 99, 22, 179, 185, 86, 166, 194, 113, 176, 26, 103, 114, 137, 127, 89, 163, 187, 153, 81, 77, 141, 91, 61, 36, 175, 78, 142, 45, 5, 177, 21, 82, 111, 8, 104, 138, 182, 139, 168, 195, 145, 15, 3, 70, 13, 132, 169, 188, 6, 147, 115, 10, 133, 122, 63, 135, 38, 1, 117, 193, 31, 54, 131, 90, 51, 121, 118, 74,

134, 11, 33, 160, 29, 0, 109, 35, 143, 106, 124, 128, 62, 18, 55, 34, 170, 152, 4, 149, 28, 20, 60, 94, 66, 47, 148, 199]

TSP A

	min	mean	max
NN to any point 2 regret	108151	117138.49	124921
NN to any point weighted 2 regret	70010	72401.245	75452
Greedy Cycle 2 regret	108804	116681.18	123447
Greedy Cycle weighted 2 regret	71108	72148.225	73718
Random	236779	264682.585	291811
NN to last point	83182	85108.51	89433
NN to any point	71179	73174.06	75450
Greedy Cycle	71488	72609.075	74410

TSP B

	min	mean	max
NN to any point 2 regret	69933	74444.46	80278
NN to any point weighted 2 regret	44891	47664.46	55247
Greedy Cycle 2 regret	65043	70264.645	76325
Greedy Cycle weighted 2 regret	47144	50997.345	56747
Random	192818	212917.165	234388
NN to last point	52319	54390.43	59030
NN to any point	44417	45870.255	53438
Greedy Cycle	48765	51301.73	57324

Conclusions

- Performance of methods with pure 2-regret heuristics is abysmal.
- 2-regret heuristics weighted with greedy methods (with 50%/50% weights) marginally outperform purely greedy methods.
- Since both algorithms have the same computational complexity their run time is very similar with greedy cycle being slightly faster in both variants.
- NN to any point gives on average slightly better results with weighted heuristic. while greedy cycle fares slightly better with pure 2 regret heuristic.